Proiect HDL
Etapa 1

top.v

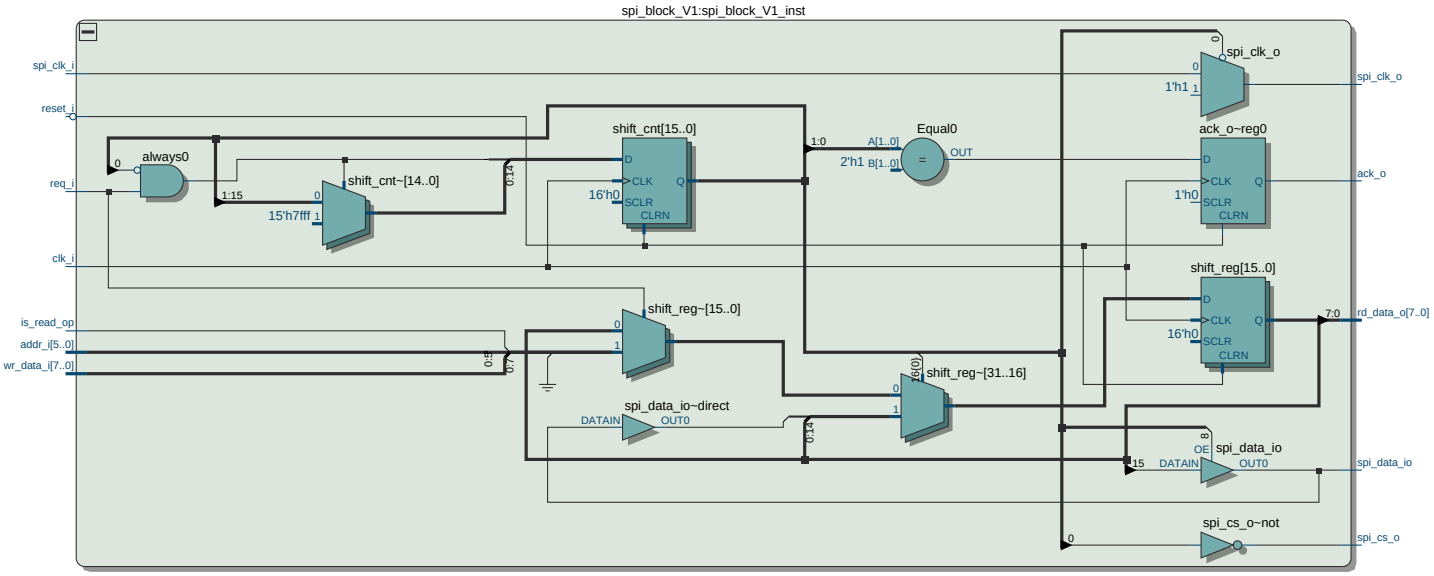
```
`timescale 1ns / 1ns

module top(
    input               MAX10_CLK1_50,

    output reg [9:0]    LEDR,
    input [9:0]         SW,
    input [1:0]         KEY,

    output              GSENSOR_CS_N,
    output              GSENSOR_SCLK,
    inout               GSENSOR_SDI
);

    wire        clk;
    wire        spi_clk;
    wire        reset;

    wire        ack_o;
    reg         ack;
    wire [7:0]  rd_data_o;

    reg         key_d;
    wire        req_i;
    wire        is_read_op;
    wire [5:0]  addr_i;
    wire [7:0]  wr_data_i;
    reg [7:0]   data;

    assign req_i        = ~key_d & KEY[1];  // negedge detector
    assign is_read_op   = SW[9];
    assign addr_i       = SW[8:3];


    // Clock generator IP
    clk_wiz clk_wiz_0(
        .areset     (~KEY[0]        ),
        .inclk0     (MAX10_CLK1_50  ),
        .c0         (clk            ),  // 2MHz
        .c1         (spi_clk        ),  // 2MHz - 220 deg offset
        .locked     (locked         )
    );

    assign reset = ~locked;


    // LEDs asignment with received data from spi
    always @(posedge clk or posedge reset) begin
        if(reset)
            LEDR <= {10{1'b0}};
        else if(ack_o)
            LEDR <= {2'b00, rd_data_o};
```

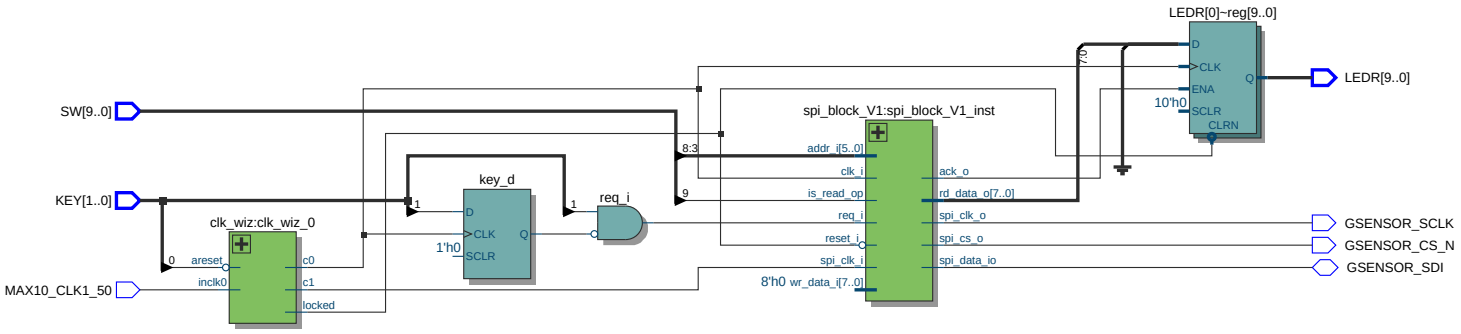
```
    end

    // button delay
    always @(posedge clk) begin
        key_d <= KEY[1];
    end

    // SPI to req_ack converter
    spi_block_V1 #(
        .TP          (1           )
    ) spi_block_V1_inst (
        .reset_i     (reset       ),
        .clk_i       (clk         ),
        .spi_clk_i   (spi_clk     ),

        // Req-ack interface
        .req_i       (req_i       ),
        .is_read_op  (is_read_op  ),
        .addr_i      (addr_i      ),
        .wr_data_i   (8'd0        ),

        .ack_o       (ack_o       ),
        .rd_data_o   (rd_data_o   ),

        // SPI interface
        .spi_cs_o    (GSENSOR_CS_N    ),
        .spi_clk_o   (GSENSOR_SCLK    ),
        .spi_data_io(GSENSOR_SDI     ),
    );

endmodule
```

spi_block_V1.v

```
module spi_block_V1 #(
    parameter TP = 1
)(
    input        reset_i,
    input        clk_i,          // max 5MHz for ADXL345 sensor
    input        spi_clk_i,      // same frequency as clk_i, 220 deg phase
offset

    // Req-ack interface
    input        req_i,          // starts the transaction
    input        is_read_op,     // is read operation - should be ready on
req_i
    input  [5:0] addr_i,         // reg address - should be ready on req_i
    input  [7:0] wr_data_i,      // input data - should be ready on req_i

    output reg   ack_o,          // transaction complete
    output [7:0] rd_data_o,      // output data - ready when ack_o

    // SPI interface
    output       spi_cs_o,
    output       spi_clk_o,
    inout        spi_data_io
);

localparam SHREG_WIDTH = 16;
```

```
reg [SHREG_WIDTH-1 : 0] shift_reg;
reg [SHREG_WIDTH-1 : 0] shift_cnt;

assign spi_cs_o  = ~shift_cnt[0];
assign spi_clk_o  = spi_cs_o ? 1'b1 : spi_clk_i;

assign spi_data_io = shift_cnt[8] ? shift_reg[SHREG_WIDTH-1] : 1'bz;
assign rd_data_o  = shift_reg[7:0];


// Shift register counter - counts up to its length
always @ (posedge clk_i or posedge reset_i) begin
    if (reset_i)
        shift_cnt <= #TP {SHREG_WIDTH{1'b0}};
    else if (req_i && spi_cs_o)
        shift_cnt <= #TP {SHREG_WIDTH{1'b1}};
    else
        shift_cnt <= #TP shift_cnt >> 1;
end


// IO shift register - for SPI bus
always @ (posedge clk_i or posedge reset_i) begin
    if (reset_i)
        shift_reg <= #TP {(SHREG_WIDTH){1'b0}};
    else if (~spi_cs_o)
        shift_reg <= #TP (shift_reg << 1) | spi_data_io;
    else if (req_i)
        shift_reg <= #TP {is_read_op, 1'b0, addr_i, wr_data_i};
end


// Acknowledge register
always @ (posedge clk_i or posedge reset_i) begin
    if (reset_i)
        ack_o <= #TP 1'b0;
    else
        ack_o <= #TP (shift_cnt[1:0] == 2'b01);
end

endmodule
```


spi_block_V1:spi_block_V1_inst
spi_clk_i
reset_i
req_i
clk_i
is_read_op
addr_i[5..0]
wr_data_i[7..0]
always0
shift_cnt~[14..0]
15'h7fff
shift_cnt[15..0]
16'h0
Equal0
2'h1
OUT
shift_reg~[15..0]
shift_reg~[31..16]
spi_data_io~direct
DATAIN
OUT0
spi_clk_o
1'h1
ack_o~reg0
1'h0
shift_reg[15..0]
spi_data_io
OE
spi_cs_o~not
spi_clk_o
ack_o
rd_data_o[7..0]
spi_data_io
spi_cs_o


LEDR[0]~reg[9..0]
D
CLK
ENA
SCLR
CLRN
Q
10'h0
LEDR[9..0]
SW[9..0]
KEY[1..0]
MAX10_CLK1_50
clk_wiz:clk_wiz_0
areset
inclk0
c0
c1
locked
key_d
D
CLK
SCLR
Q
1'h0
req_i
spi_block_V1:spi_block_V1_inst
addr_i[5..0]
clk_i
is_read_op
req_i
reset_i
spi_clk_i
8'h0 wr_data_i[7..0]
ack_o
rd_data_o[7..0]
spi_clk_o
spi_cs_o
spi_data_io
GSENSOR_SCLK
GSENSOR_CS_N
GSENSOR_SDI
8:3
9
7:0
1
0